01211373 Machine Learning and Programming for Industry

Lecture 11 Supplement 2

Where attention beats recurrence

Loop closure and route forecasting on a warehouse patrol robot

- The fixed-size state: capacity and path length
- Attention as a content-addressed lookup
- Long-range memory: has the rover been here before?
- Interpretability: attention maps, checked by occlusion
- Multi-step forecasting: attention finds the lap period
- Positions, cost, and when to keep the LSTM

Notebook: mlpi26_Lect11_nbsup2.ipynb
Companion to the CNC-cell lecture on ch. 16 and to Lecture 10 supplement 2 (rover RNNs)

THE BOTTLENECK

One state vector has to carry everything

- An LSTM must decide at every step what to keep, before it knows what will be asked. Remembering K things costs K times the room, and information from far back travels through every intermediate step.
- Self-attention keeps every token. At the end the query compares itself with all of them directly: path length one, no compression in advance.

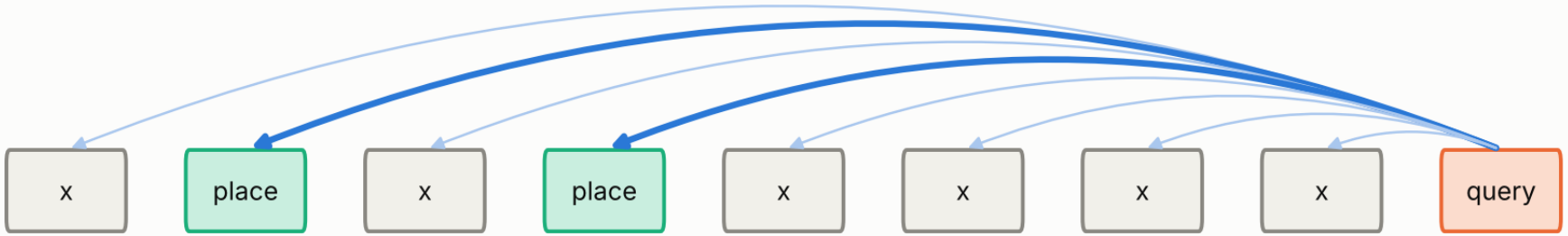
LSTM: everything the past says must squeeze through one state vector



h (96 numbers) carries everything; the path from a 'place' to the query grows with time

Transformer: the query looks every earlier token up directly

one step from anywhere, weighted by similarity of content

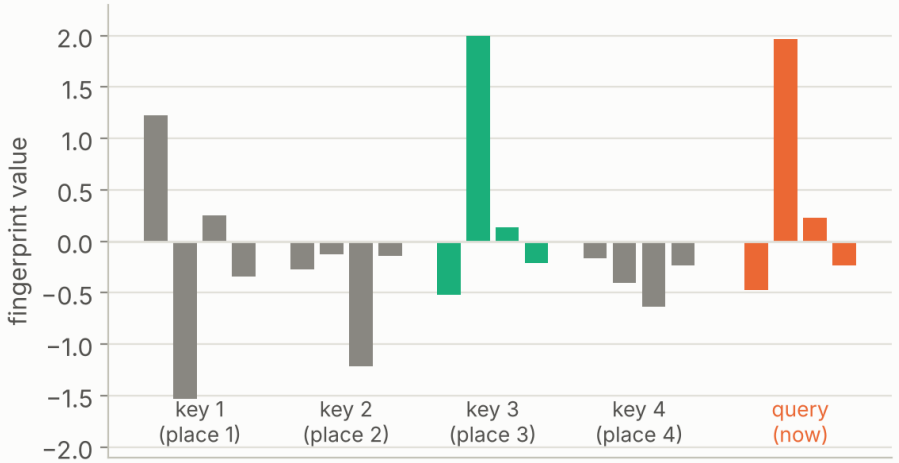


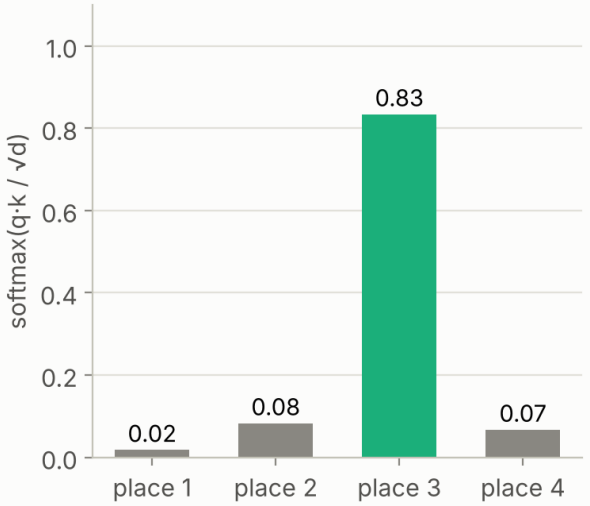
Two limits of a recurrent state: capacity (how much fits) and path length (how far it has to travel). Attention removes both and pays T^2 for it.

MECHANISM

Attention is a soft, content-addressed lookup

- Keys are what each stored token offers to be found by, values are what it hands over, the query is what the current token is looking for. Scores q·k/√d go through a softmax and mix the values.






```
scores = q @ K.T / sqrt(d)
w = softmax(scores)
out = w @ V

# PyTorch's kernel:
F.scaled_dot_product_attention(
    Q, K, V)

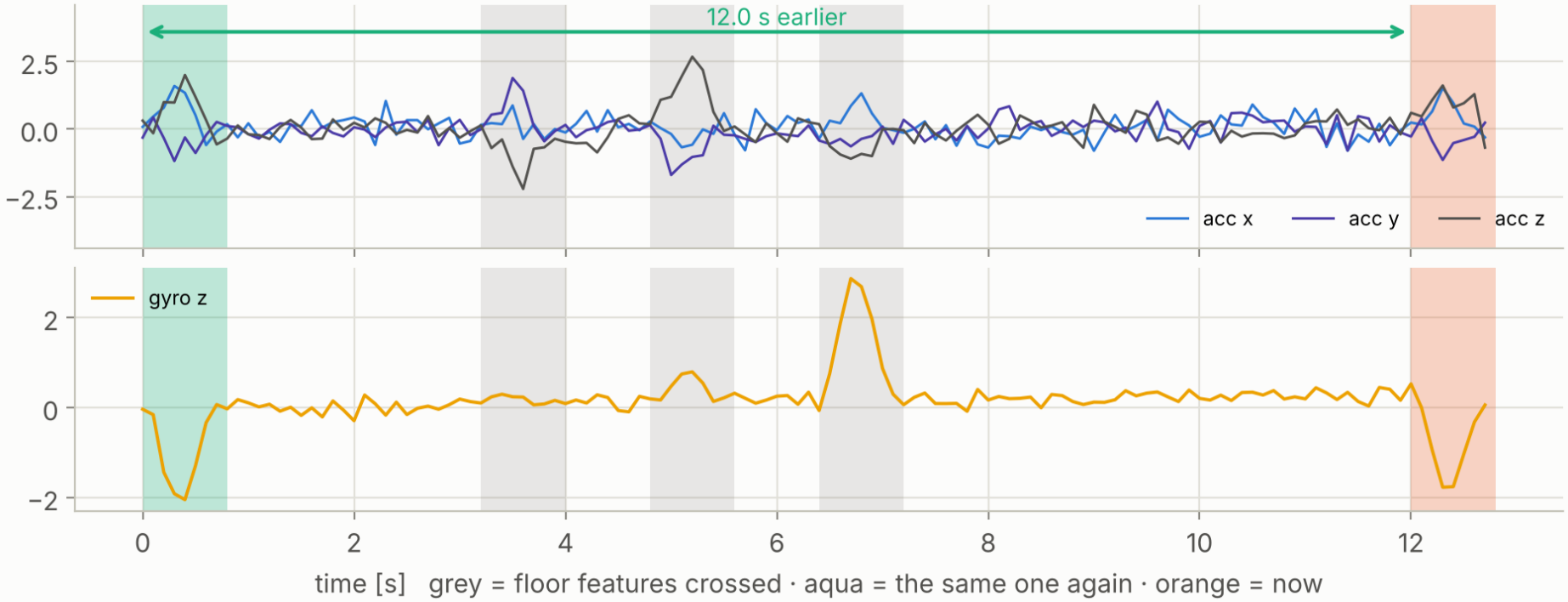
# by hand vs kernel:
# identical to 1e-7
```

Four stored fingerprints, one noisy re-observation of place 3: weights 0.02 / 0.08 / 0.83 / 0.07. The lookup is by content, not by position.

TASK 1 · LOOP CLOSURE

Has the rover been here before?

- The rover crosses K floor features in 12.8 s, each leaving a 0.8 s IMU fingerprint across four channels. At the end it crosses one more. Half the time it is a re-observation of one of the K, half the time a new one.
- A bump flag says that a feature was crossed, not which. Detection is trivial; memory and comparison are the whole task. Fresh runs are drawn every epoch, so neither model can overfit.

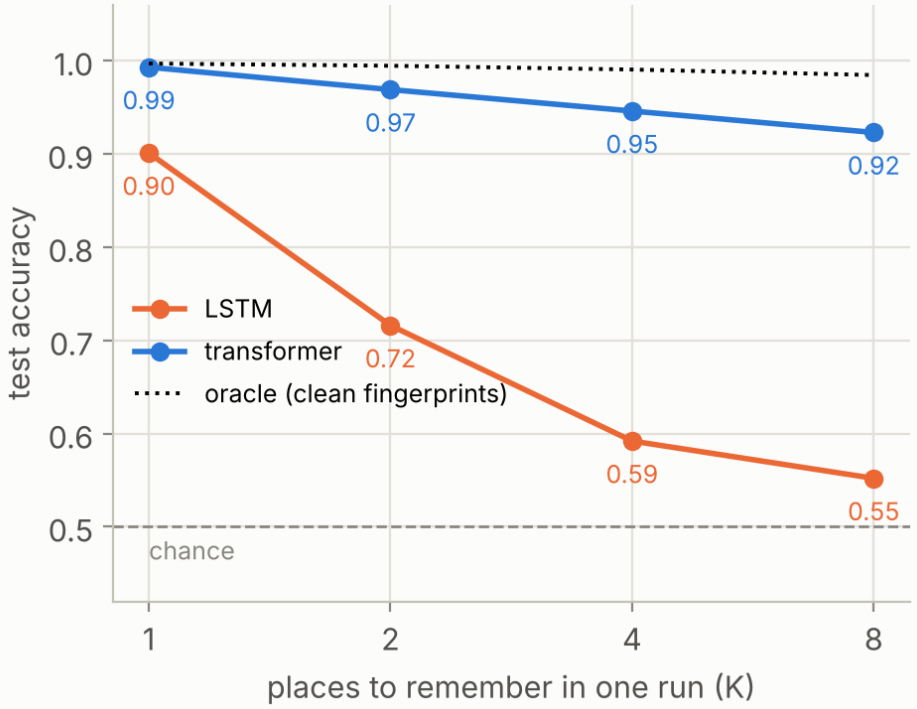


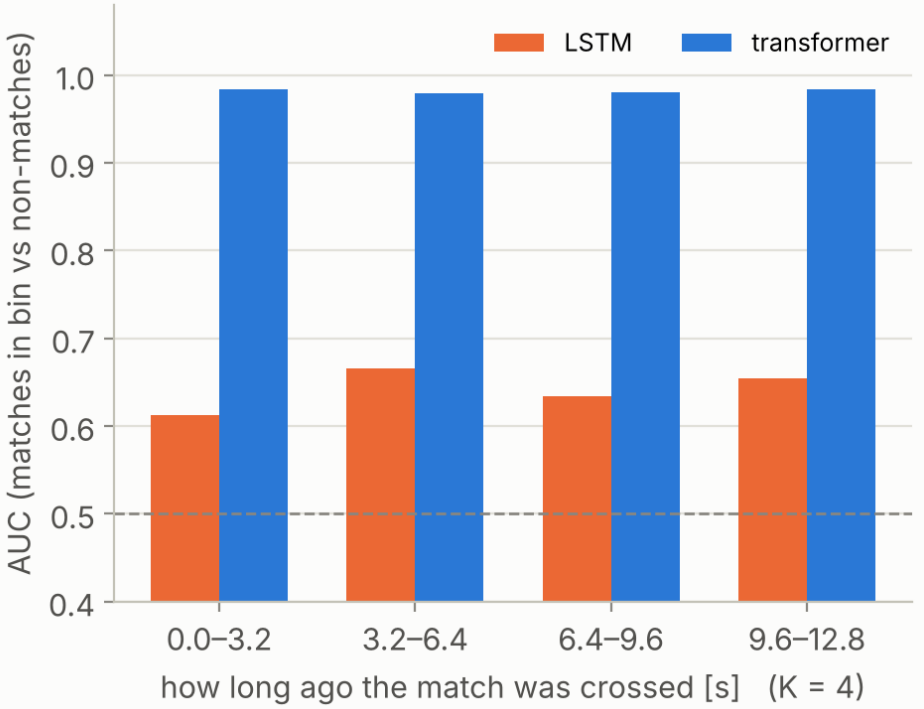
This is place recognition, the core decision of loop closure in SLAM, stripped down to one yes/no question per run.

RESULT · LONG-RANGE MEMORY

More places to remember: the LSTM breaks first

- With one place the LSTM reaches 90.1 %. At K = 4 it is at 59.2 % and at K = 8 at 55.2 %, near chance. The transformer holds 99.3 % → 92.3 %, close to the oracle ceiling.
- Right panel: at K = 4 the LSTM is poor at every gap, recent or old. Over 31 tokens, what defeats it is capacity rather than distance.



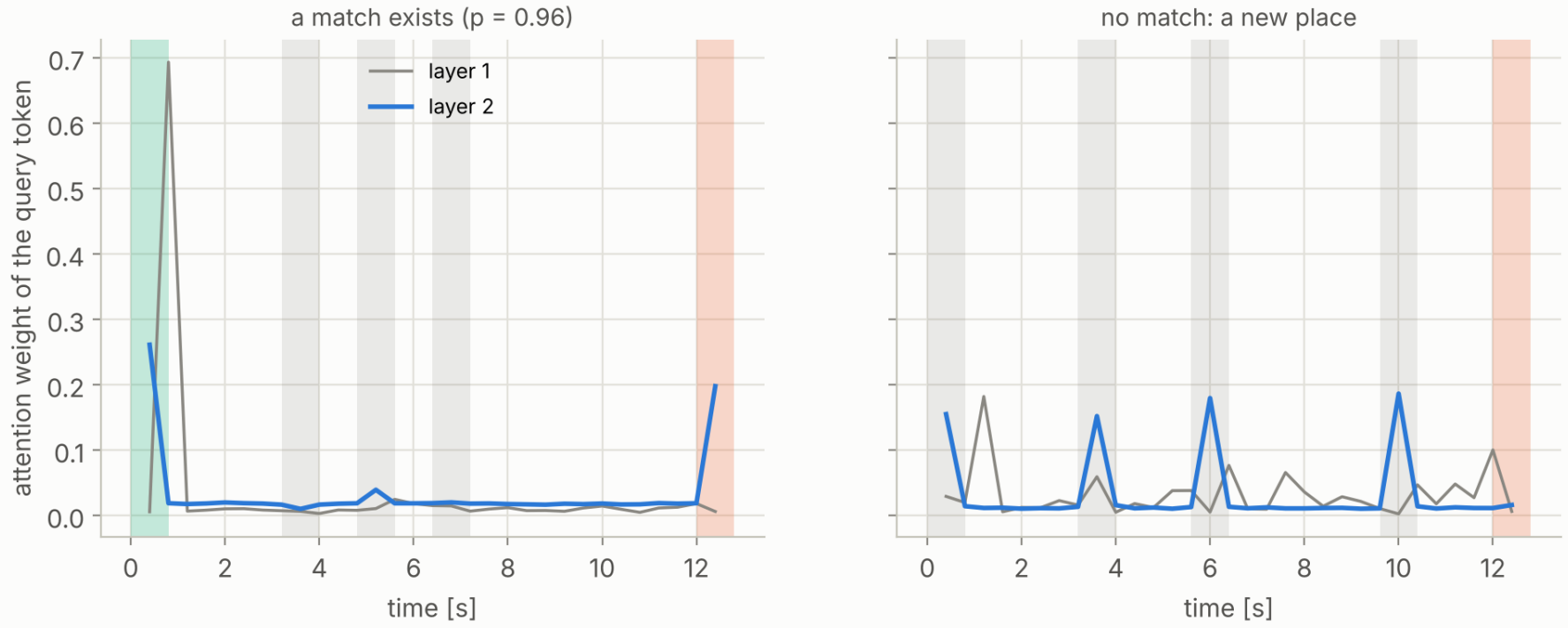


Same front end, same data, same epochs: only the time-mixing layer differs. At K = 4, 94.6 % vs 59.2 %.

INTERPRETABILITY

What the query attends to, checked by occlusion

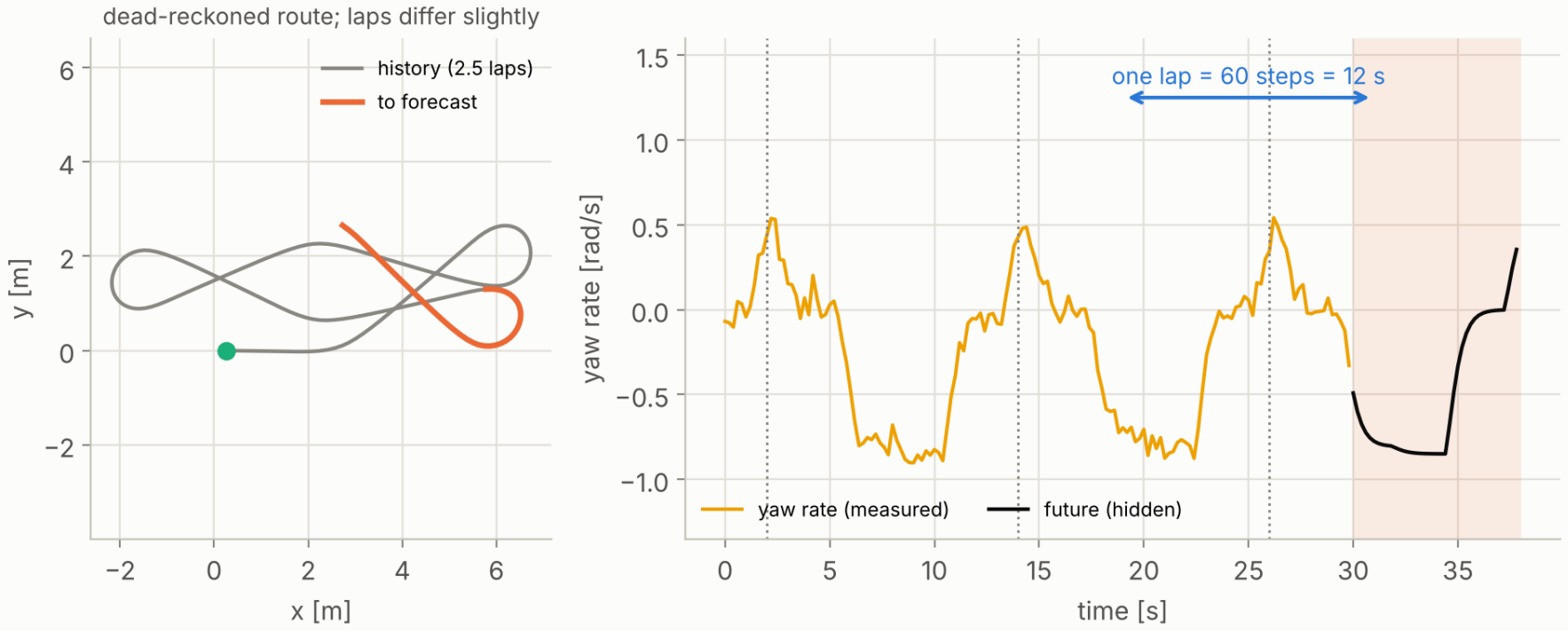
- In the last layer the query puts 28 % of its attention on the matching feature alone, about three times what each other feature gets; its peak lands on the match in 88 % of positive runs.
- Occlusion test: overwrite the matching feature with background vibration and P(seen before) drops from 0.91 to 0.12. Overwrite a different feature: 0.91.



Attention maps are hypotheses about what the model uses. Here occlusion confirms the hypothesis, which is what makes the map an explanation.

Forecasting a patrol route that repeats

- The rover repeats a random closed route on a fixed timer: one lap every 60 steps (12 s). Given 150 steps of measured yaw rate and speed, predict the next 40 steps (8 s).
- Both models get the same input: the history, then 40 blank future slots with a flag set. Both read their forecast off those slots.

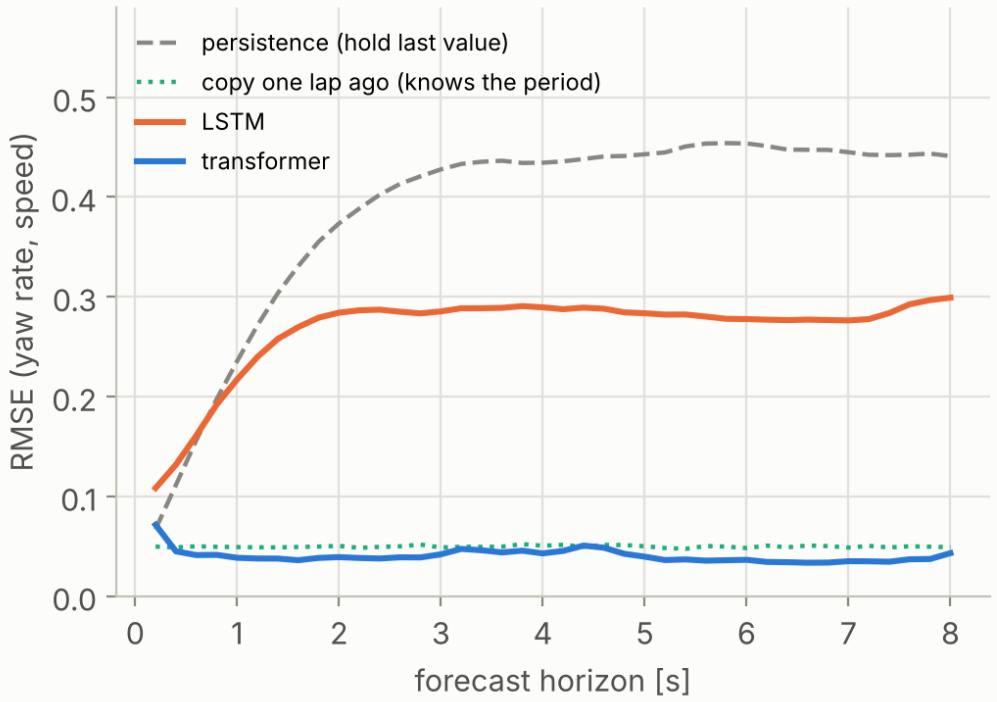


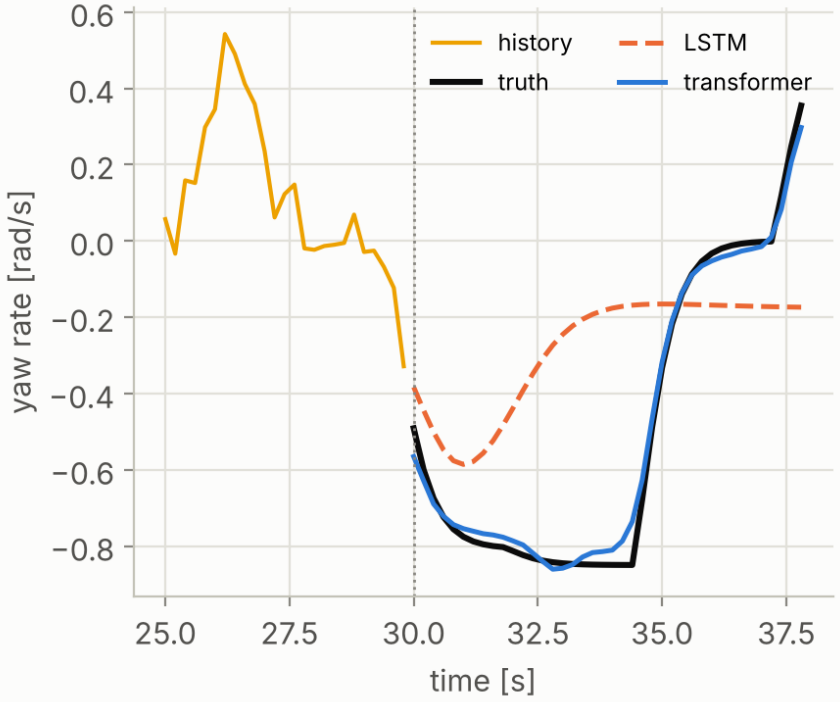
The route is new in every run, so it cannot live in the weights. It has to be read from this run's own last lap.

RESULT · MULTI-STEP FORECASTING

Forecast error against horizon

- Persistence is good for 0.2 s and useless after. The LSTM is barely better beyond the first second (0.289 at 4 s): it has learned a smooth average, not the route.
- The transformer stays near 0.043 over all 8 s, below even "copy one lap ago" (0.050), because it averages the two laps it can see.

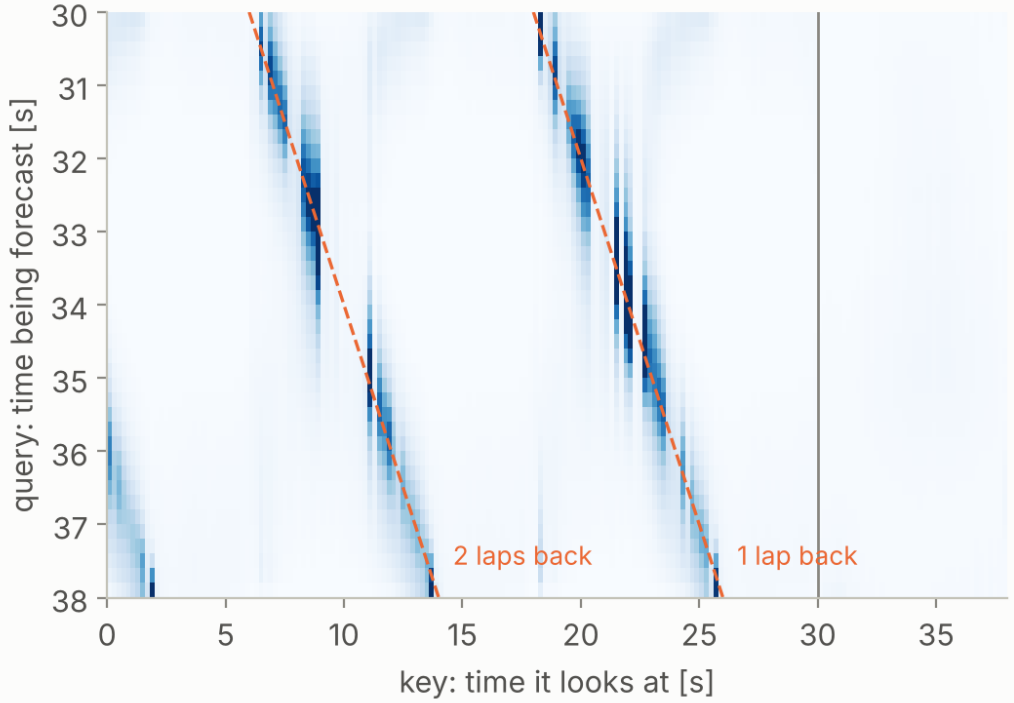


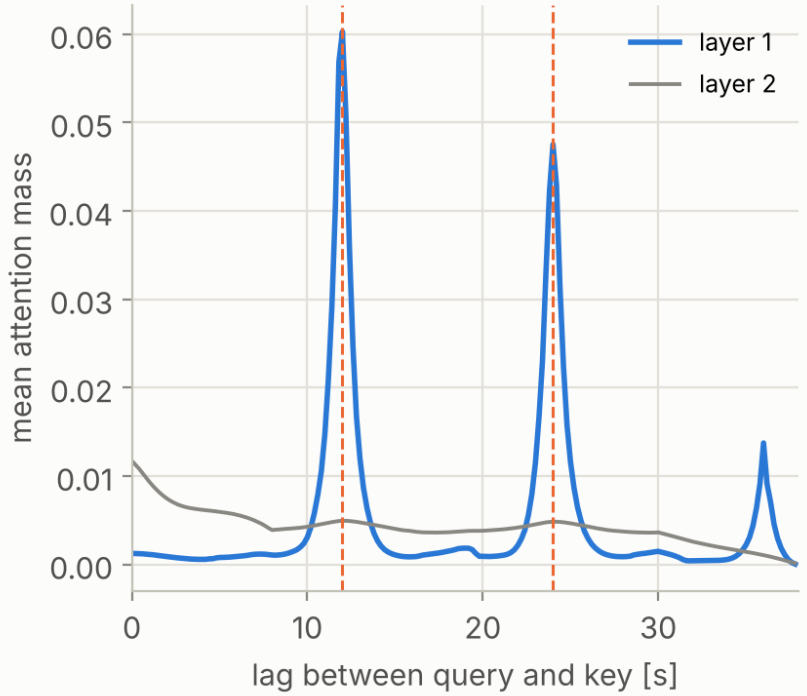


Mean RMSE over the 8 s horizon: persistence 0.388 · LSTM 0.267 · transformer 0.040 (6.7× lower).

The attention found the lap period on its own

- Average layer 1's attention over heads and 200 runs, and ask how much a future slot puts on a key k steps earlier. The peak is at 60 steps; the lap is 60.
- Without a positional embedding the 40 future slots are identical inputs, so every forecast is the same flat line: the measured spread across horizons is exactly 0.

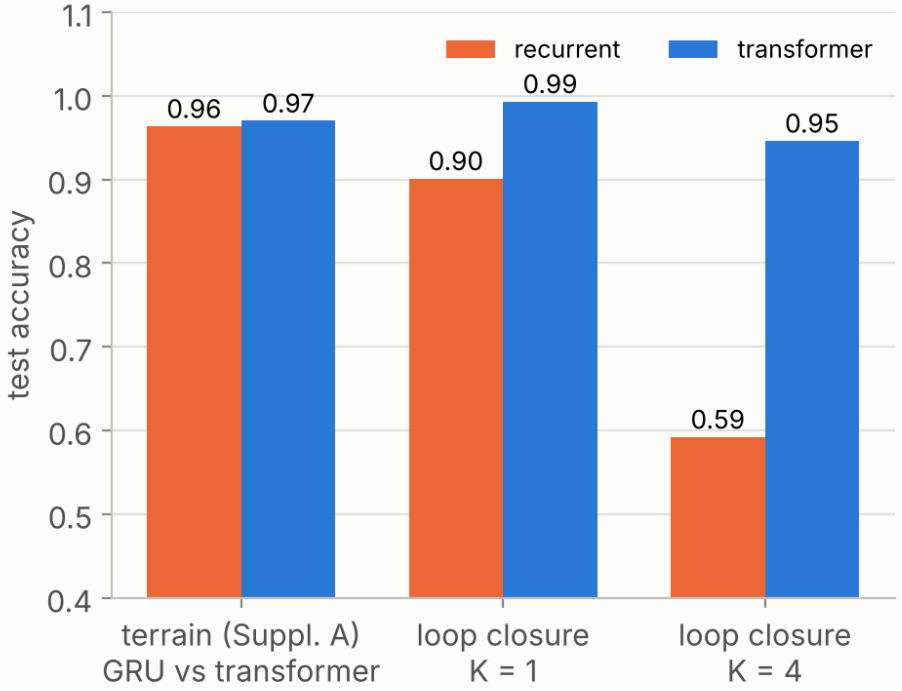


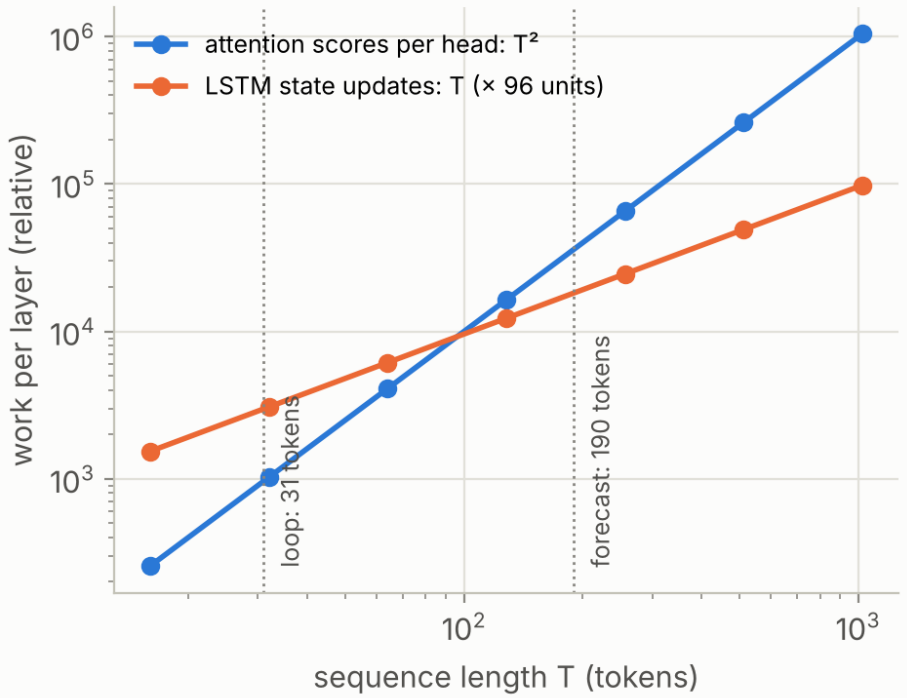


Layer 1: 24 % of attention within ±2 steps of one lap back, 20 % near two laps. Nobody told it the period.

When to keep the recurrent model

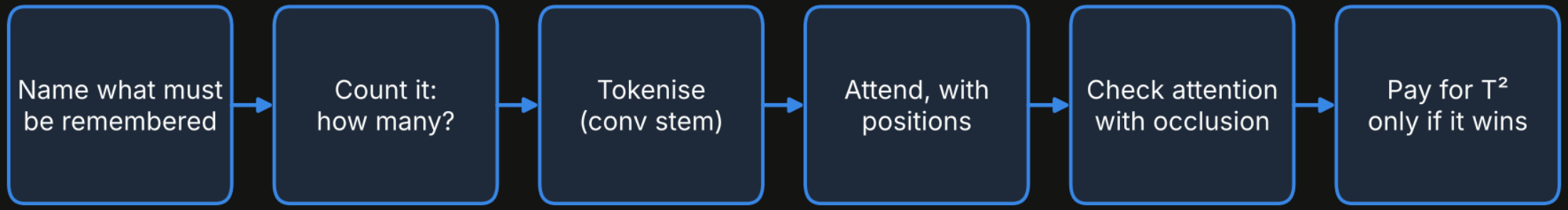
- On Supplement A's terrain task the answer is a running statistic of vibration: nothing to look up. The transformer reaches 97.0 %, the GRU 96.3 %: 0.7 points for 4.6× the parameters.
- Attention compares every token with every other: T^2 scores per head. Tokenise first (a conv stem or patches), so T stays in the hundreds.





The gain is where the answer must be looked up: +35 points on loop closure at K = 4, +1 on terrain against the GRU.

What to take away



1. Name what must be remembered and count it: one angle fits in a state; K fingerprints or a whole lap do not.
2. Attention is a content-addressed lookup: every token stays available and is one step from the query.
3. Long-range memory: at K = 4 places, 94.6 % vs 59.2 % with the same front end.
4. Forecasting: 6.7× lower error over 8 s; attention found the 60-step lap by itself.
5. Read attention maps as hypotheses and test them with occlusion.
6. Positions are part of the input. Without them the future slots are identical.
7. Pay T^2 only where something must be looked up; on running statistics keep the (right) RNN.

Generated by Claude and customized by

dew.ninja 2026

Notebook: mlpi26_Lect11_nbsup2.ipynb (8 exercises, references) · Pair with Lecture 10 Supplement 2 for the full RNN → attention arc